

Aqui está o manual revisado, com passos duplicados removidos, didática aprimorada e estrutura mais lógica:

MANUAL DO GITHUB

(Git e GitHub - Guia Prático para Iniciantes)

1. PRIMEIROS PASSOS

1.1 Configuração Inicial

```
# Configurar usuário (global)
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"
git config --global init.defaultBranch main
```

1.2 Primeiro Repositório

Passo a Passo:

1. Crie um novo repositório no GitHub (com mesmo nome do projeto local)
2. No terminal do vscode, navegue até sua pasta do projeto:

```
cd c:/pasta_do_projeto
```

3. Inicialize o Git:

```
git init
```

4. Conecte ao repositório remoto:

```
git remote add origin https://github.com/seu-usuario/nome-repositorio.git
```

2. CONTROLE DE ARQUIVOS

2.1 O que Subir para o GitHub?

- Arquivos de código, documentação e assets essenciais
- **Evite:** arquivos grandes (>100MB), sensíveis (.env) ou temporários

2.2 Usando .gitignore

Crie um arquivo `.gitignore` na raiz com:

```
# Exemplo:
node_modules/
.env
*.log
*.tmp
- Colocar no gitignore os ambientes virtuais, senhas ou outros arquivos sensíveis
```

Para aplicar:

```
git add .gitignore
git commit -m "Adiciona arquivo .gitignore"
```

3. PRIMEIRO COMMIT

3.1 Fluxo Básico

```
# 1. Adicionar arquivos específicos:
git add README.md .gitignore

# 2. Preparar commit:
git commit -m "Primeiro commit: estrutura inicial"

# 3. Enviar para o GitHub:
git push -u origin main
```

3.2 Atualizações Posteriores

```
# Adicionar alterações:  
git add pasta/arquivo.ext  
  
# Commitar:  
git commit -m "Descrição clara das alterações"  
  
# Enviar:  
git push origin main
```

4. BOAS PRÁTICAS

4.1 Mensagens de Commit

- Use verbos no imperativo (“Adiciona”, “Corrige”, “Atualiza”)
- Seja específico:
 - ❌ “Mudanças”
 - ✅ “Corrige cálculo de impostos no módulo financeiro”

4.2 Estrutura de Pastas

Sugestão para projetos de dados:

```
_projeto/  
├─ _1_dados/  
├─ _2_scripts/  
├─ _3_documentacao/  
└─ README.md
```

5. GIT LFS (PARA ARQUIVOS GRANDES)

5.1 Configuração

```
# Instalar:
git lfs install

# Rastrear tipos de arquivo:
git lfs track "*.csv" "*.zip"
- aqui você esta rastreando arquivos .csv e .zip

# Verificar:
git lfs track
```

5.2 Uso

```
git add arquivo_grande.csv
git commit -m "Adiciona dataset inicial"
git push origin main
```

6. SOLUÇÃO DE PROBLEMAS

6.1 Desfazendo Ações

```
# Modificações não commitadas:
git restore arquivo.txt

# Remover do staged:
git restore --staged arquivo.txt

# Alterar último commit:
git commit --amend -m "Nova mensagem"
```

6.2 Reiniciando Repositório

```
rm -rf .git
git init
git remote add origin URL
git add .
git commit -m "Reinício do projeto"
git push -f origin main # CUIDADO: apaga histórico remoto!
```

7. WORKFLOW AVANÇADO

7.1 Trabalhando com Branches

```
# Criar nova branch:
git checkout -b nova-feature

# Enviar para o GitHub:
git push -u origin nova-feature

# Voltar para main:
git checkout main
```

Aqui está a seção completa sobre **git reset**, organizada de forma didática para

8. GIT RESET - CONTROLE DE HISTÓRICO E ARQUIVOS

8.1 O que é git reset?

Comando para:

- Desfazer commits
- Remover arquivos da área de staging
- Restaurar o projeto para versões anteriores

⚠ **Cuidado**: Alterações podem ser permanentes. Use com precaução!

8.2 Tipos de Reset

1. Reset Soft (--soft)

- **0 que faz**: Remove commits mas mantém alterações em staging
- **Quando usar**: Para refazer mensagens de commit ou reorganizar commits pequenos

```
``bash
```

```
git reset --soft HEAD~1 # Remove o último commit, mantendo alterações prontas p
```

2. Reset Mixed (padrão)

- **O que faz:** Remove commits e tira arquivos do staging (mas mantém alterações no diretório)
- **Quando usar:** Quando você adicionou arquivos por engano ao staging

```
git reset HEAD~1          # Equivalente a git reset --mixed HEAD~1
git reset arquivo.txt      # Remove arquivo específico do staging
```

3. Reset Hard (-hard)

- **O que faz:** Apaga completamente commits e alterações não commitadas
- **Quando usar:** Para descartar totalmente alterações indesejadas

```
git reset --hard HEAD~1    # Apaga último commit E todas as alterações não salvas
git reset --hard abc123    # Volta para o commit específico (hash abc123)
```

8.3 Cenários Práticos

Cenário 1: Desfazer commit não enviado ao GitHub

```
git reset --soft HEAD~1    # Remove commit mas mantém alterações
# Edite os arquivos se necessário
git commit -m "Nova mensagem correta"
```

Cenário 2: Descartar alterações locais não commitadas

```
git reset --hard          # CUIDADO: Perde TODAS as alterações não commitadas
```

Cenário 3: Corrigir arquivos no staging

```
git add arquivo1.txt arquivo2.txt  # Adicionou por engano
git reset arquivo2.txt              # Remove apenas arquivo2.txt do staging
```

8.4 Reset vs Revert

Comando	Altera Histórico?	Ideal Para
<code>git reset</code>	✅ Sim	Correções locais (antes do push)
<code>git revert</code>	❌ Não (cria novo commit)	Correções em commits já públicos

8.5 Boas Práticas

1. **Nunca faça reset em commits já enviados ao repositório remoto** (use `git revert`)
2. **Sempre verifique o status antes:**

```
git status
git log --oneline # Veja hashes dos commits
```

3. **Para salvar alterações temporariamente** antes do reset:

```
git stash # Guarda alterações não commitadas
git reset --hard HEAD
git stash pop # Recupera alterações
```

8.6 Recuperando Commits Apagados

Se você fez reset por engano:

```
git reflog # Mostra todo o histórico de ações
git reset --hard abc123 # Volta para o commit desejado (use hash do reflog)
```

Exemplo Visual

Fluxo de alteração com diferentes tipos de reset:

```
Commit A --- Commit B (HEAD) [Estado atual]
      |
      |— soft: mantém alterações em staging
      |— mixed: mantém alterações no dir. de trabalho
      |— hard: remove tudo (volta exatamente para A)
```

Esta seção cobre desde o básico até situações complexas de recuperação, sempre alertando sobre os riscos de cada operação.

OBSERVAÇÕES IMPORTANTES

⚠️ **Armadilhas Comuns**

1. **Nunca trabalhe diretamente na branch ``main``**

.....

..... - Sempre crie branches para novas funcionalidades (``git checkout -b nova-b``)

.....

2. **Cuidado com ``git push -f``**

.....

..... - Sobrescreve o histórico remoto - pode causar perda de trabalho de outros

.....

3. **Problemas com arquivos grandes**

.....

..... - GitHub bloqueia pushes acima de 100MB - use Git LFS ou adicione ao ``.git``

.....

💡 **Dicas Valiosas**

1. **Sempre comece com ``git status``**

.....

..... - Verifique o estado atual antes de qualquer operação

.....

2. **Commite frequentemente**

.....

..... - Pequenos commits com mensagens claras > um commit gigante

.....

3. **Mantenha seu ``.gitignore`` atualizado**

.....

..... - Adicione padrões de arquivos temporários/desnecessários desde o início

.....

4. **Convenções de nomenclatura**

.....

..... - Branches: ``feat/nome-da-feature``, ``fix/correcao-x``

.....

..... - Tags: ``v1.0.0``, ``release-2023-10``

.....

🚫 **Quando Algo Der Errado

1. **Use ``git log --oneline``**


```
..... - ... Visualize o histórico de commits de forma compacta
.....

2. ... **Para desfazer quase tudo**

.....
..... bash
.....
..... Copy
.....
..... Download
.....

..... git reflog # Mostra todas as ações realizadas
..... git reset --hard [hash-do-commit] # Volta para estado anterior
.....

3. ... **Conflitos de merge?**

..... - ... Mantenha a calma! Edite os arquivos conflitantes manualmente, removendo
.....

### 📅 **Próximos Passos**

1. ... Explore recursos avançados:
.....
..... - ... `git stash` (para salvar trabalho não finalizado)
.....
..... - ... `git rebase` (para limpar histórico de commits)
.....
..... - ... GitHub Actions (automatização de workflows)
.....

2. ... Pratique com:
.....
..... - ... [GitHub Learning Lab](https://lab.github.com/)
.....
..... - ... [Oh My Git!](https://ohmygit.org/) (jogo educativo)
.....

3. ... Documente seu projeto:
.....
..... - ... Crie um README.md completo
.....
..... - ... Adicione licença (MIT, GPL, etc.)
.....

Lembre-se: Todo desenvolvedor já cometeu erros com Git - o importante é aprender
```